

MACHINE LEARNING FUNDAMENTALS SERIES
Neural Networks Pt. 2: Backpropagation Main Ideas
Optimizing Network Parameters via Chain Rule Differentiation and Gradient Descent

Source: StatQuest with Josh Starmer — Neural Networks Pt. 2: Backpropagation Main Ideas

Crucial Pedagogical Declaration: The "Main Ideas" Focus

Backpropagation is a comprehensive algorithm, but its foundational concepts are best understood through a focused lens. To prevent getting lost in multi-parameter indices and complex notation, this document demonstrates backpropagation with respect to a **single parameter**: the final output bias term b_3 .

We assume that all other connection weights (w_1, w_2, w_3, w_4) and hidden node biases (b_1, b_2) have already been pre-optimized (indicated in **green**). This isolates the core mechanics of backpropagation:

1. Applying the **Chain Rule** to compute the exact gradient of the Loss function with respect to b_3 .
2. Plugging this derivative into **Gradient Descent** to iteratively update b_3 until convergence.

1. Problem Formulation & Network Architecture

Consider the drug dosage effectiveness dataset where dosage $x \in [0, 1]$ predicts effectiveness $y \in \{0, 1\}$:

- **Low Dosage** ($x_1 = 0.0$): Ineffective ($y_1 = 0.0$).
- **Medium Dosage** ($x_2 = 0.5$): Highly Effective ($y_2 = 1.0$).
- **High Dosage** ($x_3 = 1.0$): Ineffective ($y_3 = 0.0$).

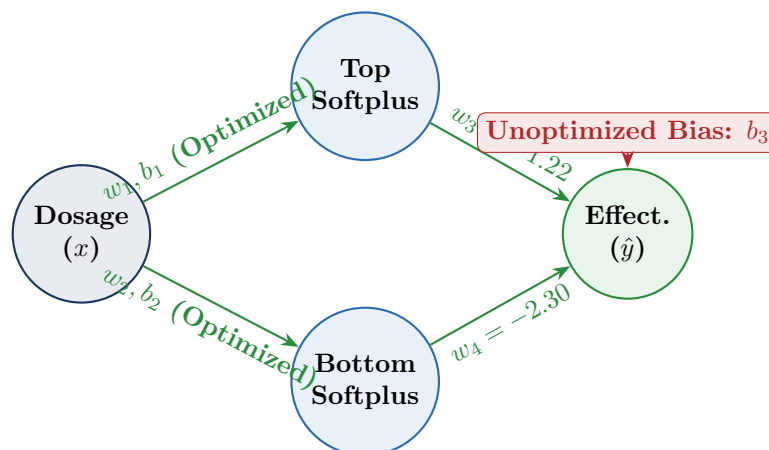


Figure 1: Demonstration Neural Network. Connection weights (w_1, w_2, w_3, w_4) and hidden biases (b_1, b_2) are pre-optimized (green). Only the output bias b_3 (red) is unoptimized and targeted by backpropagation.

The predicted output $\hat{y}_i$ is the sum of the blue curve (top node), orange curve (bottom node), and final bias b_3 :

$$\hat{y}_i = \text{Curve}_{\text{blue}}(x_i) + \text{Curve}_{\text{orange}}(x_i) + b_3 \quad (1)$$

2. Step 1: Chain Rule Derivation for b_3

To optimize b_3 , we measure fit quality using the **Sum of Squared Residuals (SSR)** as our Loss function:

$$\text{SSR} = \sum_{i=1}^n (\text{Observed}_i - \text{Predicted}_i)^2 = \sum_{i=1}^3 (y_i - \hat{y}_i)^2 \quad (2)$$

We compute the derivative of SSR with respect to b_3 , written as $\frac{\partial \text{SSR}}{\partial b_3}$, using the **Chain Rule**:

$$\frac{\partial \text{SSR}}{\partial b_3} = \frac{\partial \text{SSR}}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial b_3} \quad (3)$$

2.1. Part A: Derivative of Loss with Respect to Prediction ($\frac{\partial \text{SSR}}{\partial \hat{y}}$)

Applying the power rule and chain rule to the outer squared function:

$$\begin{aligned} \frac{\partial \text{SSR}}{\partial \hat{y}} &= \frac{\partial}{\partial \hat{y}} \left[\sum_{i=1}^3 (y_i - \hat{y}_i)^2 \right] \\ &= \sum_{i=1}^3 2(y_i - \hat{y}_i) \cdot \frac{\partial (y_i - \hat{y}_i)}{\partial \hat{y}_i} \\ &= \sum_{i=1}^3 2(y_i - \hat{y}_i) \cdot (-1) = -2 \sum_{i=1}^3 (y_i - \hat{y}_i) \end{aligned} \quad (4)$$

2.2. Part B: Derivative of Prediction with Respect to Bias ($\frac{\partial \hat{y}}{\partial b_3}$)

Differentiating the prediction equation with respect to b_3 :

$$\frac{\partial \hat{y}_i}{\partial b_3} = \frac{\partial}{\partial b_3} [\text{Curve}_{\text{blue}}(x_i) + \text{Curve}_{\text{orange}}(x_i) + b_3] = 0 + 0 + 1 = 1 \quad (5)$$

Note: Blue and orange curves depend solely on $w_1, w_2, w_3, w_4, b_1, b_2$ and are independent of b_3 .

2.3. Combined Gradient Formula

Multiplying Part A and Part B together yields the final gradient formula:

$$\frac{\partial \text{SSR}}{\partial b_3} = \left[-2 \sum_{i=1}^3 (y_i - \hat{y}_i) \right] \times 1 = -2 \sum_{i=1}^3 (y_i - \hat{y}_i) \quad (6)$$

3. Step 2: Gradient Descent Optimization Loop for b_3

With the derivative formula established, we initialize $b_3^{(0)} = 0$ and use **Gradient Descent** with learning rate $\eta = 0.1$.

3.1. Iteration 1 ($b_3^{(0)} = 0$)

At $b_3 = 0$, running dosages $x_1 = 0.0, x_2 = 0.5, x_3 = 1.0$ through the network yields:

- $\hat{y}_1 = -2.60 \implies \text{Residual}_1 = 0.0 - (-2.60) = +2.60$
- $\hat{y}_2 = -1.61 \implies \text{Residual}_2 = 1.0 - (-1.61) = +2.61$
- $\hat{y}_3 = -2.61 \implies \text{Residual}_3 = 0.0 - (-2.61) = +2.61$

Initial Loss: $\text{SSR} = (2.60)^2 + (2.61)^2 + (2.61)^2 = \mathbf{20.4}$.

Gradient (slope) at $b_3 = 0$:

$$\text{Slope} = -2 \times (2.60 + 2.61 + 2.61) = -2 \times 7.82 = \mathbf{-15.7} \quad (7)$$

Step Size and updated parameter:

$$\text{Step Size} = \text{Slope} \times \eta = -15.7 \times 0.1 = \mathbf{-1.57} \quad (8)$$

$$b_3^{(1)} = b_3^{(0)} - \text{Step Size} = 0 - (-1.57) = \mathbf{1.57} \quad (9)$$

3.2. Iteration 2 ($b_3^{(1)} = 1.57$)

Increasing b_3 to 1.57 shifts the prediction curve up by +1.57, shrinking residuals:

- New Loss: $\text{SSR} = \mathbf{7.8}$ (reduced from 20.4).
- New Gradient: $\text{Slope} = \mathbf{-6.26}$.
- New Step Size: $-6.26 \times 0.1 = \mathbf{-0.626}$.
- Updated Bias: $b_3^{(2)} = 1.57 - (-0.626) = \mathbf{2.19}$.

3.3. Convergence to Optimal b_3^*

Iterative steps shrink the step size towards 0, reaching the minimum loss at:

$$b_3^* = \mathbf{2.61} \implies \text{Minimum Loss SSR} = \mathbf{0.46} \quad (10)$$

4. Summary of Backpropagation Mechanics

Table 1: Step-by-Step Backpropagation Workflow for Single Parameter Optimization

Phase	Core Operation & Mathematical Expression
1. Forward Pass	Run inputs x_i through network to calculate predicted outputs $\hat{y}_i$.
2. Loss Calculation	Quantify prediction error: $\text{SSR} = \sum_{i=1}^n (y_i - \hat{y}_i)^2$.
3. Chain Rule	Compute parameter gradient: $\frac{\partial \text{SSR}}{\partial b_3} = \frac{\partial \text{SSR}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial b_3} = -2 \sum_{i=1}^n (y_i - \hat{y}_i)$.
4. Step Size	Multiply gradient by learning rate: $\text{Step Size} = \text{Slope} \times \eta$.
5. Parameter Update	Update parameter: $b_3^{\text{new}} = b_3^{\text{old}} - \text{Step Size}$ until $\text{Step Size} \approx 0$.

Core Takeaway

Backpropagation combines two fundamental ideas: **Chain Rule Differentiation** to find how the loss function changes relative to any parameter, and **Gradient Descent** to step the parameter towards the loss minimum. While demonstrated here for a single parameter b_3 , the exact same principle extends simultaneously to all weights and biases across arbitrary network depths.